



College of Engineering, Construction and Living Sciences
Bachelor of Information Technology
ID608001: Intermediate Application Development Concepts
Level 6, Credits 15
Project

Assessment Overview

In this **individual** assessment, you will design and develop **two** games.

Learning Outcomes

At the successful completion of this course, learners will be able to:

1. Apply design patterns and programming principles using software development best practices.
2. Design and implement full-stack applications using industry relevant programming languages.

Assessments

Assessment	Weighting	Due Date	Learning Outcomes
Practical	20%	Sunday 9th November at 11.59 PM	1
Project	80%	Sunday 9th November at 11.59 PM	1 and 2

Conditions of Assessment

You will complete this assessment mostly during your learner-managed time. However, there will be time during class to discuss the requirements and your progress on this assessment. This assessment will need to be completed by **Sunday 9th November at 11.59 PM**.

Pass Criteria

This assessment is criterion-referenced (CRA) with a cumulative pass mark of **50%** over all assessments in **ID608001: Intermediate Application Development Concepts**.

Authenticity

All parts of your submitted assessment **must** be completely your work. Do your best to complete this assessment without using AI tools. You need to demonstrate to the course lecturer that you can meet the learning outcomes for this assessment.

Learning to use AI tools is an important skill. While AI tools are powerful, you **must** be aware of the following:

- If you provide an AI tool with a prompt that is not refined enough, it may generate a not-so-useful response
- Do not trust the AI tool's responses blindly. You **must** still use your judgement and may need to do additional research to determine if the response is correct
- Acknowledge what AI tool you have used. In the assessment's repository **README.md** file, please include what prompt(s) you provided to the AI tool and how you used the response(s) to help you with your work

It also applies to code snippets retrieved from **StackOverflow** and **GitHub**.

Failure to do this may result in a mark of **zero** for this assessment.

Policy on Submissions, Extensions, Resubmissions and Resits

The school's process concerning submissions, extensions, resubmissions and resits complies with **Otago Polytechnic** policies. Learners can view policies on the **Otago Polytechnic** website located at <https://www.op.ac.nz/about-us/governance-and-management/policies>.

Submission

You **must** submit all files via **GitHub**. Create a repository and add the course lecturer as a collaborator. The latest files will be used to mark against the marking rubric. Please test before you submit. Partial marks **will not** be given for incomplete functionality. Late submissions will incur a **10% penalty per day**, rolling over at **12.00 AM**.

Extensions

Familiarise yourself with the assessment due date. Extensions will **only** be granted if you are unable to complete the assessment by the due date because of **unforeseen circumstances outside your control**. The length of the extension granted will depend on the circumstances and **must** be negotiated with the course lecturer before the assessment due date. A medical certificate or support letter may be needed. Extensions will not be granted on the due date and for poor time management or pressure of other assessments.

Resits

Resits and reassessments **are not** applicable in **ID608001: Intermediate Application Development Concepts**.

Functionality - Learning Outcomes 1, 2 (25%)

Note: If you wish to develop different games than those specified below, you **must** get approval from the course lecturer first.

- **Godot 2D Platformer** game with the following functionality:
 - Player movement using keyboard input.
 - Platform collision system, i.e., solid, moving and one-way platforms.
 - Health system.
 - Win and lose conditions.
 - Three distinct levels with increasing difficulty.
 - Hazards and obstacles, i.e., spikes, traps or pits.
 - Three distinct enemies with different behaviours.
 - Double jump ability.
 - Collectible items, i.e., coins or power-ups.
 - Scoring system.
 - Choose five features of your choice. These features should be different from the ones listed above.
- **3D Unity/Godot Obstacle Dodging** game with the following functionality:
 - Player movement using keyboard or mouse input.
 - Obstacle collision system, i.e., solid and moving obstacles.
 - Health system.
 - Five distinct obstacles with different behaviours.
 - Two distinct levels with increasing difficulty.
 - Collectible items, i.e., coins or power-ups.
 - Scoring system.
 - Camera effects, i.e., screen shake.
 - Choose five features of your choice. These features should be different from the ones listed above.
- Deploy both games to **Itch.io**.

Code Quality and Best Practices - Learning Outcomes 1, 2 (40%)

- Write clean, readable, and maintainable code.
- Use modular, reusable components and avoid code duplication.
- Keep code simple and easy to understand.
- Optimise performance by managing resources efficiently.

Version Control - Learning Outcome 1 (5%)

- Regular commits are made throughout development, demonstrating meaningful progress.
- Commit messages are clear and descriptive.
- The repository shows a clean and logical commit history that reflects the development process of the games.

Reflection - Learning Outcomes 1, 2 (5%)

- Write a short reflection (300-500 words) on your development process and learning throughout the project, including:
 - Challenges you faced and how you overcame them.
 - What you would do differently in future projects.
 - New skills, tools or practices you learned during the project.

Presentation - Learning Outcomes 1, 2 (5%)

- Record a 5-10 minute screen recording of your **Godot 2D Platformer** and **3D Unity/Godot Obstacle Dodging** games.
- Demonstrate the games' features.
- Make sure the video is clear and easy to follow.
- Submit a link to the presentation in your repository's **README.md** file.

Marking Rubric

Functionality - Learning Outcomes 1, 2 (25%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Godot 2D Platformer - Player Movement and Input (2%)	Player movement using keyboard input works flawlessly. Smooth and responsive controls. Movement feels polished and natural.	Player movement functional with keyboard input. Generally responsive with minor control issues.	Basic player movement implemented. Some responsiveness issues or control limitations.	Player movement non-functional or severely limited. Poor or no keyboard input handling.
Godot 2D Platformer - Platform System (2%)	All three platform types fully functional (solid, moving, one-way). Collision detection works perfectly. Platforms behave as expected.	Platform types mostly functional. Collision detection works with minor issues. Most platforms behave correctly.	Basic platform collision implemented. Some platform types missing or partially functional.	Platform collision system non-functional or missing. Platforms don't work as intended.
Godot 2D Platformer - Health, Win/Lose Conditions (2%)	Health system fully implemented and displayed clearly. Win and lose conditions work correctly. Clear feedback to player.	Health system functional with minor display issues. Win/lose conditions mostly work. Some feedback present.	Basic health system present. Win/lose conditions partially implemented. Limited feedback.	Health system or win/lose conditions missing or non-functional. No player feedback.
Godot 2D Platformer - Levels and Difficulty (2%)	Three distinct levels implemented with clear progression. Difficulty increases appropriately. Levels are well-designed and balanced.	Three levels present with some difficulty progression. Most levels functional with minor design issues.	2-3 levels implemented. Limited difficulty progression. Basic level design.	Fewer than 2 levels or non-functional. No difficulty progression. Poor level design.
Godot 2D Platformer - Hazards, Obstacles and Enemies (2%)	Hazards and obstacles fully functional and well-placed. Three distinct enemies with clearly different behaviours working perfectly.	Hazards and obstacles mostly functional. Three enemies present with some behavioural differences. Minor issues.	Basic hazards and obstacles present. 2-3 enemies with limited behavioural variety.	Hazards/obstacles missing or non-functional. Fewer than 2 enemies or no distinct behaviours.
Godot 2D Platformer - Mechanics and Systems (2%)	Double jump ability works flawlessly. Collectible items fully functional. Scoring system accurately tracks and displays score. All systems well-integrated.	Double jump functional with minor issues. Collectibles mostly work. Scoring system present with some accuracy issues.	Basic double jump implemented. Some collectibles functional. Basic scoring system present.	Double jump missing or non-functional. Collectibles or scoring system missing or broken.

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Godot 2D Platformer - Custom Features (2%)	Five custom features fully implemented and functional. Features are creative, well-integrated and enhance gameplay.	4-5 custom features implemented with minor issues. Features add value to gameplay.	3-4 custom features present with functionality gaps. Basic integration.	Fewer than 3 custom features or non-functional. Poor integration or no gameplay value.
3D Obstacle Dodging - Player Movement and Input (2%)	Player movement using keyboard or mouse input works flawlessly. Controls are smooth, responsive and intuitive.	Player movement functional. Controls generally responsive with minor issues.	Basic player movement implemented. Some control responsiveness issues.	Player movement non-functional or severely limited. Poor or no input handling.
3D Obstacle Dodging - Obstacle System (2%)	Five distinct obstacles with clearly different behaviours fully functional. Collision system works perfectly for solid and moving obstacles.	Five obstacles present with some behavioural differences. Collision system mostly functional with minor issues.	3-4 obstacles implemented with limited variety. Basic collision system present.	Fewer than 3 obstacles or no distinct behaviours. Collision system non-functional or missing.
3D Obstacle Dodging - Health and Levels (1%)	Health system fully implemented and displayed clearly. Two distinct levels with appropriate difficulty increase.	Health system functional. Two levels present with some difficulty progression. Minor issues.	Basic health system present. 1-2 levels with limited difficulty progression.	Health system missing or non-functional. Fewer than 1 level or no difficulty progression.
3D Obstacle Dodging - Systems and Effects (2%)	Collectible items fully functional. Scoring system accurate. Camera effects (screen shake) implemented effectively and enhance gameplay.	Collectibles mostly work. Scoring system present with minor issues. Camera effects functional.	Basic collectibles present. Basic scoring system. Camera effects partially implemented.	Collectibles or scoring missing or non-functional. No camera effects or broken implementation.
3D Obstacle Dodging - Custom Features (2%)	Five custom features fully implemented and functional. Features are creative, well-integrated and enhance gameplay.	4-5 custom features implemented with minor issues. Features add value to gameplay.	3-4 custom features present with functionality gaps. Basic integration.	Fewer than 3 custom features or non-functional. Poor integration or no gameplay value.
Deployment (2%)	Both games successfully deployed to Itch.io. Games are accessible, playable and perform well. Professional presentation on platform.	Both games deployed to Itch.io with minor issues or performance concerns. Adequate presentation.	Both games deployed but with accessibility or functionality issues. Basic presentation.	Games not deployed, deployment non-functional, or only one game deployed.

Code Quality and Best Practices - Learning Outcomes 1, 2 (40%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Code Readability and Maintainability (10%)	Code is exceptionally clean and readable with consistent formatting. Meaningful variable and function names. Well-organised file and scene structure. Comprehensive comments where needed.	Code is clean and readable with good naming conventions. Organised structure with some helpful comments. Minor inconsistencies.	Code is mostly readable. Some unclear naming or inconsistent formatting. Basic organisation present.	Code is difficult to read with poor naming conventions. Inconsistent or chaotic structure.
Modularity and Reusability (10%)	Excellent use of modular, reusable scripts and scenes. Components are well-abstracted. No code duplication. DRY principles followed throughout.	Good modular structure with reusable components. Minimal code duplication. Most code follows DRY principles.	Some modular components present. Some code duplication exists. Partial adherence to DRY principles.	Little modularity. Significant code duplication. Poor component abstraction.
Code Simplicity and Clarity (10%)	Code is simple, elegant and easy to understand. Complex logic is well-explained. Appropriate use of engine features without over-engineering.	Code is generally simple and clear. Most logic is easy to follow with minor complex sections.	Code works but contains unnecessarily complex sections. Some logic is difficult to follow.	Code is overly complex or convoluted. Difficult to understand the implementation.
Performance and Resource Management (10%)	Excellent resource management. Efficient use of nodes and scenes. Proper memory management. No performance bottlenecks. Optimised game loop and physics.	Good resource management. Mostly efficient node usage. Adequate optimization with room for improvement. Minor performance considerations.	Basic resource management. Some inefficient implementations or optimization gaps. Performance acceptable but not optimised.	Poor resource management. Inefficient use of engine features. Missing optimization. Performance issues present. Memory leaks or frame drops.

Version Control - Learning Outcome 1 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Commit History and Messages (5%)	Regular, meaningful commits throughout development. Clear, descriptive commit messages. Logical progression showing development process. Clean commit history.	Regular commits with mostly clear messages. Commit history shows good development progression with minor gaps.	Commits present but irregular or clustered. Some commit messages unclear. Basic development history visible.	Few commits or poor timing. Unclear messages (e.g., "update", "fix"). No clear development progression.

Reflection - Learning Outcomes 1, 2 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Reflection Quality (5%)	Thoughtful reflection (300-500 words) covering all required points. Clear articulation of challenges, solutions and learning. Demonstrates deep understanding and self-awareness.	Good reflection covering all required points. Adequate discussion of challenges and learning with some depth. Within word count.	Reflection present covering most points. Brief or superficial discussion. May be slightly outside word count.	Reflection missing, incomplete or significantly outside word count. Lacks depth or doesn't cover required points.

Presentation - Learning Outcomes 1, 2 (5%)

Criteria	Excellent (A)	Good (B)	Satisfactory (C)	Not Yet Achieved (D)
Video Demonstration (5%)	Clear 5-10 minute recording demonstrating all Godot 2D Platformer and 3D Obstacle Dodging game features. Well-paced, professional presentation. Easy to follow with good audio/visual quality. Link submitted correctly.	Good recording within time limit. Most features demonstrated. Generally clear with minor audio/visual issues. Link submitted.	Recording present but may be too short/long. Some features demonstrated. Quality acceptable but could be clearer.	Recording missing, significantly outside time limit, unclear, or doesn't demonstrate key features. Poor quality.